strax + amstrax 3: the XAMS reconstruction stream and event tables

Explainer — from redax chunks to the event tables, where every plugin enters, what the tables hold and where the old event_info columns went (amstrax docs/overview-note-amstrax3 3ec34eb 2026-11-06, generated from the source; 2026-11-09)

In one sentence. strax is a generic engine: it knows how to chain plugins, stream data in time chunks and store results under a hash of everything that made them. **amstrax** is the XAMS part: the plugins (what happens to pulses, peaks and events), the contexts (which plugins, where data lives, the run database) and the settings (run doc, corrections from amstrax_files). You ask for one data type; strax works out and runs everything below it.

1. strax in seven words

word	meaning	in amstrax
plugin	Python class: **depends_on** (inputs), **provides** (output name), **dtype** (columns), **compute()** (the work), options as class attributes	19 plugin classes in amstrax/plugins/
data type	named product, a numpy structured array; every row has time and endtime (ns since 1970)	raw_records, records, peaks, peak_basics, events, sites, …
data kind	data types with the same rows (one per pulse / peak / event); a plugin that depends on several of the same kind gets them merged column-wise	'peaks' = peaks + peak_basics + peak_positions
context	registry of plugins + settings + storage list; st.get_array / st.make go through it	contexts.py:62 xams(), :119 context_for_daq_reader(), :221 xams_led()
option	setting of a plugin (default in the class, overridden with st.set_config); tracked options enter the lineage	XAMSConfig (xams_config.py:34): also file://, rundoc:// values
lineage	hash of plugin name + version + tracked options + lineage of the inputs: same hash = same data, change anything → new hash → recomputed, never mixed	data folders --
storage	where strax looks for / writes data, in order; save_when decides what is written	RunDB (rundb.py:178, run doc data[]) + DataDirectory folders

2. What happens when you ask for a data type

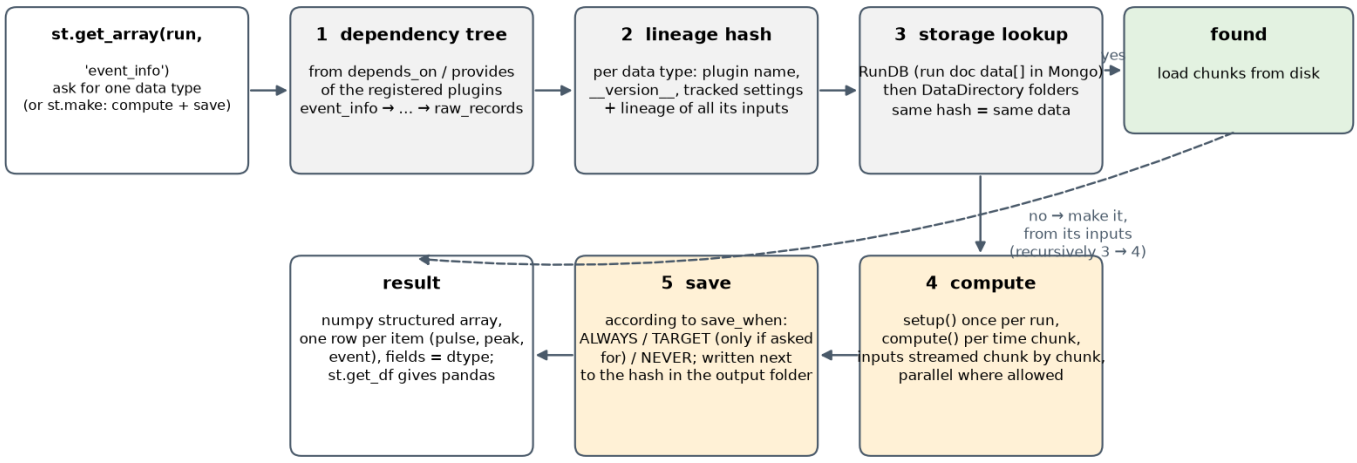


Figure 1. One request, generic strax. The dashed arrow: after computing (and saving) you get the same array as when it was loaded. Only what is missing is computed; a changed setting or plugin version makes a new lineage, so the old data is not used for it.

3. How the central processing on STBC runs it

auto_processing_new/process.py (main :558) runs per run, as xamsdata. **Stage 1** (:370): **raw_st.make(run, 'raw_records')** in the DAQ-reader context — only there may the DAQReader run; it reads the live data. **Stage 2** (:474): **st.make(run, target)** for the other targets in the main context; raw_records are only read there (forbid_creation_of). Targets come from the command line (online and offline: peak_basics events sites event_peaks peak_context event_cuts); peak_basics_ext / peak_basics_sipm are added when the run's DAQ configuration enables those channels. **Corrections**: --corrections_version (:537, default None) picks amstrax_files/corrections/_global/_global_<version>.json; apply_global_correction_version (contexts.py:272) turns every entry into st.set_config. Without it, gain_to_pe_array stays None → to_pe = 1 → peak areas in ADC·samples, not PE. Since 2026-10-18 the online loop runs with **PROD0** (approved gains + position map): runs with approved gains get PE; later runs (null gains) stay in ADC·samples. Positions come from the map in both cases. The SR-2026.3 science runs 7762–8109 are reprocessed with **PROD5** (2026-11-06: amstrax 3.1, corrections v10, cut references; table in section 7).

4. The reconstruction stream

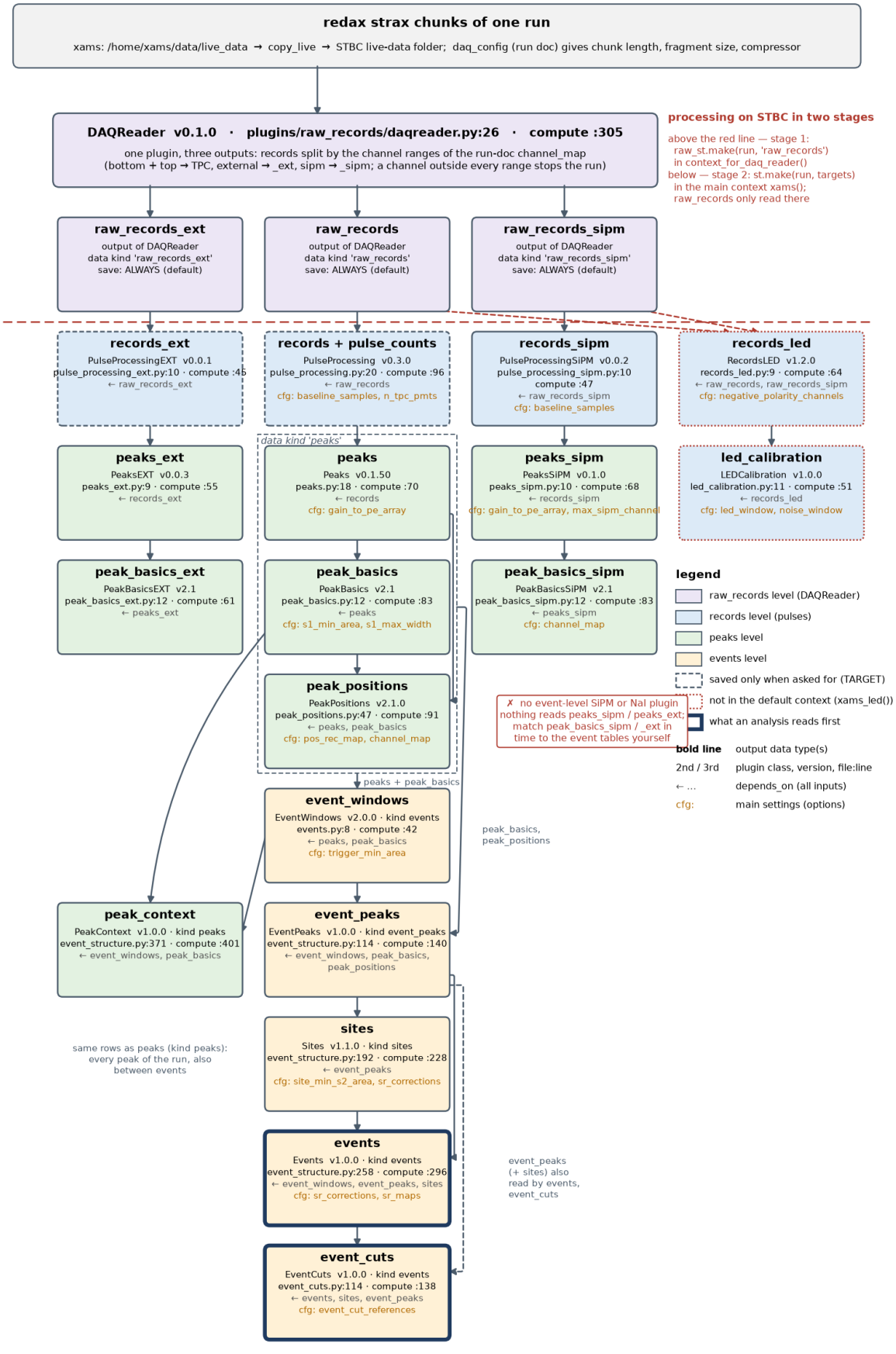


Figure 2. Every amstrax plugin and where it enters. Columns: external PMT (NaI) | TPC PMTs (main chain) | SiPMs | LED calibration. Each box is one plugin; its text is read from the source. Arrows show the main flow; the full input list is the ← line in each box. Dashed frame: one data kind (same rows, more columns); the event-level boxes name their data kind. Thick frame: what an analysis reads first. Registered in the default context: contexts.py:26.

5. Main chain, stage by stage

Paths relative to amstrax/amstrax/. The line numbers are those of the checkout named on page 1.

#	output (save)	plugin · file:line	depends_on	main settings (default)	what it does
1	**raw_records_sipm + raw_records_ext + raw_records** ALWAYS (default)	**DAQReader** v0.1.0 plugins/raw_records/daqreader.py:26 compute :305	(files)	channel_map = run doc daq_chunk_duration = int(5000000000.0)	Reads the redax chunk files of one run, splits the records by channel range (run-doc channel_map) into TPC / external / SiPM and makes the time absolute.
2	**records + pulse_counts** records: TARGET, pulse_counts: ALWAYS	**PulseProcessing** v0.3.0 plugins/records/pulse_processing.py:20 compute :96	raw_records	baseline_samples = 20 n_tpc_pmts = None	Keeps TPC channels; raw_to_records; baseline per channel from the first baseline_samples, flips the negative PMT pulses; integrates each pulse; counts pulses per channel.
3	**peaks** ALWAYS (default)	**Peaks** v0.1.50 plugins/peaks/peaks.py:18 compute :70	records	gain_to_pe_array = None peak_gap_threshold = 300 peak_min_pmts = 1 peak_min_area = 10	find_hits (samples above threshold) → find_peaks (hits closer than peak_gap_threshold, ≥ peak_min_pmts, area × to_pe) → sum_waveform → split_peaks → widths.
4	**peak_basics** ALWAYS (default)	**PeakBasics** v2.1 plugins/peaks/peak_basics.py:12 compute :83	peaks	s1_min_area = 10 s1_max_width = 225 s2_min_area = 10 s2_min_width = 225 channel_map = run doc	Area, widths, area_fraction_top, n_channels, saturation; type 1 = S1, 2 = S2, 0 = unknown from the width / area / AFT / channel cuts.
5	**peak_positions** ALWAYS (default)	**PeakPositions** v2.1.0 plugins/peaks/peak_positions.py:47 compute :91	peaks, peak_basics	pos_rec_map = DEFAULT_POS_REC_MAP channel_map = run doc	u, v from the top-array light fractions; x, y, r from pos_rec_map (default: latest map), with or without gains (without: ~2 mm worse); NaN without a map. No polynomial since 2.0.0.
6	**event_windows** ALWAYS (default)	**EventWindows** v2.0.0 plugins/events/events.py:8 compute :42	peaks, peak_basics	trigger_min_area = 10 left_event_extension = int(500000.0) right_event_extension = int(50000.0)	Triggers = S2s with area > trigger_min_area; window [t − left_event_extension, t + right_event_extension]; overlapping windows merged (find_peak_groups). Only time windows (event_number, time, endtime). Called 'events' until amstrax 2.x.
7	**event_peaks** ALWAYS (default)	**EventPeaks** v1.0.0 plugins/events/event_structure.py:114 compute :140	event_windows, peak_basics, peak_positions	–	Every peak fully inside an event window: peak_basics columns + x, y, with event_number, peak_index (row in the run), rank by area within its type and dt_s1 (time after the main S1 = largest S1 before the largest S2).
8	**sites** ALWAYS (default)	**Sites** v1.1.0 plugins/events/event_structure.py:192 compute :228	event_peaks	site_min_s2_area = 300 sr_corrections = None sr_maps = None elife = 30000 drift_time_cathode = 39500	Every S2 of event_peaks above site_min_s2_area: drift time, x, y, z, r, cS2 (lifetime + S2_top map, radial factor), cS2 top / bottom, rank (1 = main S2), in_drift. Corrections options from _Corrections.
9	**events** ALWAYS (default)	**Events** v1.0.0 plugins/events/event_structure.py:258 compute :296	event_windows, event_peaks, sites	sr_corrections = None sr_maps = None s1_naive_z_correction = [-52, 0, 700, -7.69]	The event as a whole: counts of peaks / S1s / S2s / sites, cS1 (S1 / cs2_top-weighted response over the in-drift sites), sums of cS2 over the in-drift sites, energy, time to previous / next event.

#	output (save)	plugin · file:line	depends_on	main settings (default)	what it does
10	**peak_context** ALWAYS (default)	**PeakContext** v1.0.0 plugins/events/event_structure.py:371 compute :401	event_windows, peak_basics	–	Every peak of the run (also between events) with the last S2 > 1 kPE and > 10 kPE before it: time since, area, event. For delayed electrons and photoionisation.
11	**event_cuts** ALWAYS (default)	**EventCuts** v1.0.0 plugins/events/event_cuts.py:114 compute :138	events, sites, event_peaks	event_cut_references = None	One boolean per cut per event (basic, isolated S1 / S2, S1 AFT and S2 width bands, cut C, fiducial, all) from events + sites + event_peaks, references per run set from amstrax_files.

6. Side branches

#	output (save)	plugin · file:line	depends_on	main settings (default)	what it does
1	**records_ext** TARGET	**PulseProcessingEXT** v0.0.1 plugins/records_ext/pulse_processing_ext.py:10 compute :45	raw_records_ext	–	Baseline + integration for the external channel (NaI).
2	**peaks_ext** ALWAYS (default)	**PeaksEXT** v0.0.3 plugins/peaks_ext/peaks_ext.py:9 compute :55	records_ext	–	Hit and peak finding on records_ext.
3	**peak_basics_ext** ALWAYS (default)	**PeakBasicsEXT** v2.1 plugins/peaks_ext/peak_basics_ext.py:12 compute :61	peaks_ext	–	Peak properties of the external peaks.
4	**records_sipm** ALWAYS (default)	**PulseProcessingSiPM** v0.0.2 plugins/records_ext/pulse_processing_sipm.py:10 compute :47	raw_records_sipm	baseline_samples = 20	Baseline + integration for the SiPM channels.
5	**peaks_sipm** ALWAYS (default)	**PeaksSiPM** v0.1.0 plugins/peaks_ext/peaks_sipm.py:10 compute :68	records_sipm	gain_to_pe_array = None max_sipm_channel = 7 channel_map = run doc	Hit and peak finding on records_sipm (area_per_channel up to max_sipm_channel).
6	**peak_basics_sipm** ALWAYS (default)	**PeakBasicsSiPM** v2.1 plugins/peaks_ext/peak_basics_sipm.py:12 compute :83	peaks_sipm	channel_map = run doc	Peak properties of the SiPM peaks.
7	**records_led** TARGET	**RecordsLED** v1.2.0 plugins/led_calibration/records_led.py:9 compute :64	raw_records, raw_records_sipm	negative_polarity_channels = (0, 1, 2, 3, 4, 5)	LED window cut from raw_records + raw_records_sipm; flips only negative_polarity_channels.
8	**led_calibration** ALWAYS (default)	**LEDCalibration** v1.0.0 plugins/led_calibration/led_calibration.py:11 compute :51	records_led	led_window = (80, 110) noise_window = (0, 10)	Amplitude and area per LED trigger (led_window vs noise_window) → input for the gain fits.

7. Where the settings come from (later overrides earlier)

#	source	what / where
1	plugin default	default= of each option in the plugin class (column 'main settings' above)
2	context	XAMS_COMMON_CONFIG in contexts.py (now only n_tpc_pmts = 5); without a run database (init_rundb=False) also the default channel_map
3	run doc	options with a rundoc:// default read the run's own value: channel_map from xams_bookkeeping.channel_map, else daq_config.channel_map, else the XAMS default (xams_config.py:119). Plugins: DAQReader, PeakBasics, PeakBasicsSiPM, PeakPositions, PeaksSiPM. DAQReader also gets chunk length, fragment size, compressor and threads from daq_config.
4	corrections_version	_global_<version>.json in amstrax_files: file:// entries are fetched per run (run ranges inside the file), numbers set directly. Table below: PROD5.
5	st.set_config(…)	your own overrides in an analysis (e.g. a smaller peak gap for single photons) → own lineage, own data folder

setting	value in _global_PROD5.json	used by
gain_to_pe_array	gain_to_pe_array_v8.json	Peaks, PeaksEXT, PeaksSiPM
elife	elife_v4.json	Events, Sites
pos_rec_map	pos_rec_map_v2.json	PeakPositions
s1_min_area	10	PeakBasics, PeakBasicsSiPM
s1_min_width	10	PeakBasics, PeakBasicsEXT, PeakBasicsSiPM
s1_max_width	150	PeakBasics, PeakBasicsEXT, PeakBasicsSiPM
s1_min_channels	3	PeakBasics, PeakBasicsSiPM
s1_max_area_fraction_top	1.0	PeakBasics, PeakBasicsSiPM
s2_min_channels	2	PeakBasics, PeakBasicsSiPM
s2_min_width	200	PeakBasics, PeakBasicsEXT, PeakBasicsSiPM
left_event_extension	100000	EventWindows
right_event_extension	50000	EventWindows
trigger_min_area	200	EventWindows
sr_corrections	sr_corrections_v2.json	Events, Sites
sr_maps	sr_maps_v2.json	Events, Sites
drift_time_gate	drift_time_gate_v3.json	Events, Sites
drift_time_cathode	drift_time_cathode_v3.json	Events, Sites
gate_cathode_distance	51.8	Events, Sites
event_cut_references	event_cut_references_v1.json	EventCuts

8. Where the SiPM and NaI data are

Up to peaks only. raw_records_sipm → records_sipm → peaks_sipm → peak_basics_sipm, and the same for the external NaI (_ext). No plugin reads them at event level. Match peak_basics_sipm / peak_basics_ext in time to the event tables (event_peaks, sites) in your own analysis, or rebuild from raw_records_sipm. The plugins read the run's own channel_map (both SiPM channels, 6 and 7, since PR #340, 2026-10-17).

9. Things to know

• **Positions**: the (u, v) → (x, y) map per peak (peak_positions; PROD5: pos_rec_map_v2.json); NaN only for runs without a map. Without gains (u, v) are in ADC·samples (~2 mm median shift).
• **Event window**: triggers are S2s above trigger_min_area; plugin defaults left 500 µs / right 50 µs, PROD5 100000 / 50000 ns with trigger_min_area = 200 PE. Delayed light up to ms after an S2 lies outside the events: peak_context has it.
• **Polarity**: PulseProcessing flips every TPC channel (baseline_per_channel(flip=True)); the polarity registers of the run are not read.
• **records are not kept**: PulseProcessing saves records only as a target (pulse_counts always). Anything above records needs the raw_records again if it must be recomputed.
• **Renewing**: python docs/notes/make_amstrax_note.py after a change to amstrax — names, versions, lines, dependencies and the columns of the event tables follow; a new or removed plugin stops the script until it gets a place and a description.

10. The event tables (amstrax 3)

Since amstrax 3.0 (deployed 2026-11-06) the event level is four linked tables on top of the event windows, plus one table of cut booleans. Each holds one kind of object: the event, the interaction site (S2), the peak. Nothing is copied between them; they link by **event_number** and **peak_index** (the row of the peak in the run's peaks, counted from the first peak). The old wide row per event (event_info) is gone; **amstrax.main_interaction(st, run)** makes its equivalent as a join.

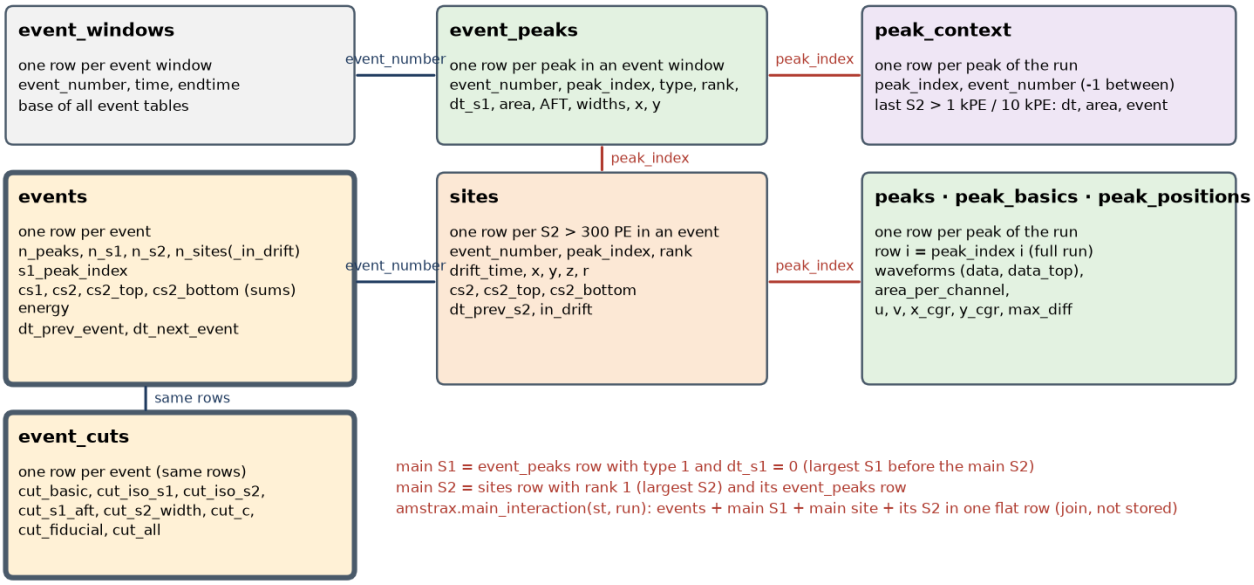


Figure 3. The event tables and their keys. events and event_cuts have the same rows (data kind events), so one st.get_df(run, ['events', 'event_cuts']) loads both; sites, event_peaks and peak_context are their own data kinds and are loaded separately and joined by key.

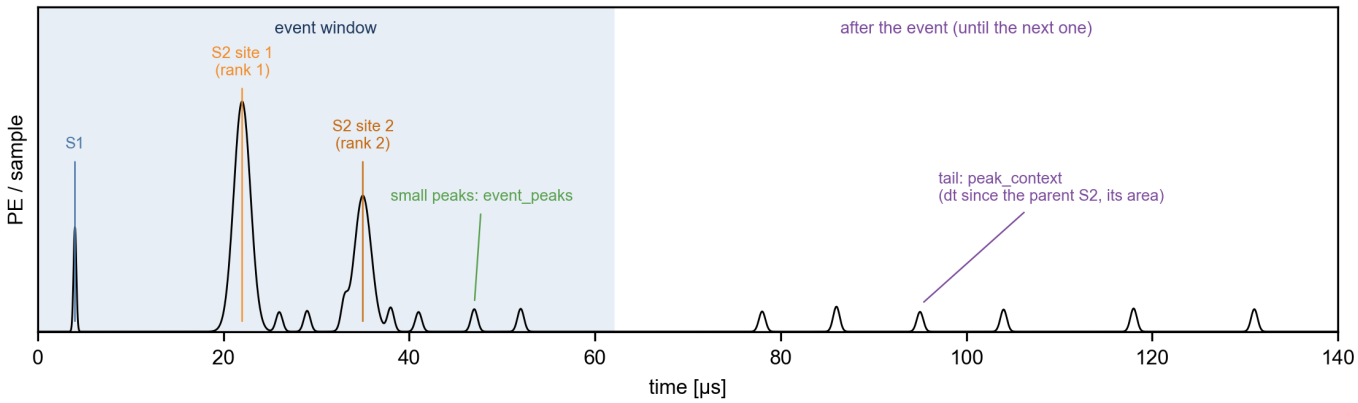


Figure 4. Where the peaks of an event go (sketch): the S1 and every peak in the window are rows of event_peaks; every S2 above 300 PE is also a site (rank 1 = main S2); peaks after the window, until the next event, are only in peak_context, which links every peak to the last large S2 before it.

table	one row per	made from	use it for
events	event	event_windows, event_peaks, sites	spectra, rates; cs1 and the sums of cs2 over the in-drift sites; energy = W (cs1/g1 + cs2_top/g2_top)
sites	S2 above site_min_s2_area (300 PE) in an event	event_peaks + corrections	position and corrected S2 of every interaction; multiple scatters; the main site (rank 1)
event_peaks	peak inside an event window	event_windows, peak_basics, peak_positions	properties of the main S1 / S2 and of every other peak of the event (rank, dt_s1)
peak_context	peak of the run (also between events)	event_windows, peak_basics	delayed electrons, photoionisation: time since and area of the last S2 > 1 kPE / > 10 kPE
event_cuts	event (same rows as events)	events, sites, event_peaks + cut references	cut_all and the single cuts; rejected events stay in the data
event_windows	event	peaks, peak_basics	only where events start and stop; not an analysis table

Definitions. Main S2 = largest S2 of the event (site of rank 1). Main S1 = largest S1 before the main S2 (event_peaks row with type 1 and dt_s1 = 0). dt_s1 = center_time minus that of the main S1; for a site it is the drift time. A site is in_drift if 0 ≤ drift_time ≤ 1.05 × drift_time_cathode; only those enter the sums of events. Fake sites (split S2s, photoionisation, trains) are kept: on science run 007790 about 40 % of the events with a site have two or more in-drift sites; select single scatters with cut_iso_s2 (in cut_all) or n_sites_in_drift == 1.
Corrections (Sites, Events; options of _Corrections, the names of the former corrected_areas). Mode 1, when sr_corrections and sr_maps are set (PROD5): cs2 = S2 · exp(drift_time / τ(t)) / f2(x, y) with the electron lifetime τ of the run, the S2_top (x, y) map and the radial S2_top factor where the maps hold one; cs1 = S1 / response, the S1 (x, y) map × S1(drift time) averaged over the in-drift sites with weight cs2_top; energy if the run is calibrated. Mode 2 (no maps): cs2 = S2 · exp(drift_time / elife), cs1 with the naive linear z correction. cs2_top = cs2 · AFT, cs2_bottom = cs2 − cs2_top. For an event with one site the values equal those of the former event_info (validated on 65 runs).

Cuts (event_cuts): cut_basic (main S1 and main S2 > 300 PE), cut_iso_s1 (alternative S1 < max(10 % of the main S1, 30 PE)), cut_iso_s2 (no other S2 ≥ 5 % of the main S2), cut_s1_aft and cut_s2_width (3σ bands against drift time), cut_c (odd-event cut: S2 AFT and width against references), cut_fiducial, cut_all. References per run set in amstrax_files (event_cut_references); has_references is False outside a calibrated set and then the reference cuts are False.

11. Columns of the event tables

From st.data_info of this checkout. Times in ns since 1970, widths in ns, areas in PE, positions in mm.

event_windows

column	type	meaning
event_number	int64	Event number in this dataset
time	int64	Event start time in ns since the unix epoch
endtime	int64	Event end time in ns since the unix epoch

events

column	type	meaning
time	int64	Start time since unix epoch [ns]
endtime	int64	Exclusive end time since unix epoch [ns]
event_number	int64	Event number in this dataset
n_peaks	int32	Number of peaks in the event
n_s1	int32	Number of S1s
n_s2	int32	Number of S2s
n_sites	int32	Number of sites (S2 > site_min_s2_area)
n_sites_in_drift	int32	Number of sites inside the drift region
s1_peak_index	int64	peak_index of the main S1; -1 if none
cs1	float32	Main S1 corrected with the cs2_top-weighted response of the sites [PE]
cs2	float32	Sum of cs2 over the in-drift sites [PE]
cs2_top	float32	Sum of cs2_top over the in-drift sites [PE]
cs2_bottom	float32	Sum of cs2_bottom over the in-drift sites [PE]
energy	float32	W (cs1 / g1 + cs2_top / g2_top) [keV]; NaN if not calibrated
area_before_main_s2	float32	Sum of the peak areas before the main S2 [PE]
large_s2_before_main_s2	float32	Largest S2 before the main S2 [PE]
dt_prev_event	int64	Start of this event minus end of the previous one [ns]; -1 for the first
dt_next_event	int64	Start of the next event minus end of this one [ns]; -1 at a chunk end

sites

column	type	meaning
time	int64	Start time since unix epoch [ns]
endtime	int64	Exclusive end time since unix epoch [ns]
event_number	int64	Event number in this dataset
peak_index	int64	Row of the S2 in peaks (link to event_peaks)
rank	int16	Rank by S2 area in this event (1 = main S2)
drift_time	float32	S2 center_time minus that of the main S1 [ns]
x	float32	x-position [mm]
y	float32	y-position [mm]
z	float32	Depth [mm] (0 at the gate)
r	float32	Radius [mm]
cs2	float32	Corrected S2, total [PE]
cs2_top	float32	Corrected S2 seen by the top array [PE]
cs2_bottom	float32	Corrected S2 seen by the bottom array [PE]
dt_prev_s2	float32	Time since the previous S2 of any size in this event (center times) [ns]; NaN if none
in_drift	bool	0 <= drift_time <= 1.05 drift_time_cathode (enters the event sums)

event_peaks

column	type	meaning
time	int64	Start time since unix epoch [ns]
endtime	int64	Exclusive end time since unix epoch [ns]
event_number	int64	Event number in this dataset
peak_index	int64	Row of the peak in peaks (counted from the first peak of the run)
type	int16	Peak type (1 S1, 2 S2, 0 other)
rank	int16	Rank by area within the type in this event (1 = largest)
dt_s1	float32	center_time minus that of the main S1 [ns]; NaN without S1
center_time	int64	Weighted center time since unix epoch [ns]
area	float32	Area, uncorrected [PE]
area_fraction_top	float32	Fraction of the area seen by the top array
range_50p_area	float32	Width, 50% area [ns]
range_90p_area	float32	Width, 90% area [ns]
rise_time	float32	Time between 10% and 50% area quantiles [ns]
n_channels	int16	Number of contributing PMTs
n_hits	int32	Number of hits contributing at least one sample to the peak
max_pmt	int16	PMT that contributes the most PE
max_pmt_area	float32	Area in the largest-contributing PMT [PE]
tight_coincidence	int16	Number of PMTs with hits within tight range of mean
n_saturated_channels	int16	Number of saturated channels
x	float32	x-position from the pos_rec_map [mm]
y	float32	y-position from the pos_rec_map [mm]

peak_context

column	type	meaning
time	int64	Start time since unix epoch [ns]
endtime	int64	Exclusive end time since unix epoch [ns]
peak_index	int64	Row of the peak in peaks (counted from the first peak of the run)
event_number	int64	Event that contains the peak; -1 between events
dt_prev_s2_1k	int64	Time since the last S2 > 1000 PE (center times) [ns]; -1 if none
area_prev_s2_1k	float32	Area of the last S2 > 1000 PE [PE]
event_prev_s2_1k	int64	Event of the last S2 > 1000 PE; -1 if between events
dt_prev_s2_10k	int64	Time since the last S2 > 10000 PE (center times) [ns]; -1 if none
area_prev_s2_10k	float32	Area of the last S2 > 10000 PE [PE]
event_prev_s2_10k	int64	Event of the last S2 > 10000 PE; -1 if between events

event_cuts

column	type	meaning
time	int64	Start time since unix epoch [ns]
endtime	int64	Exclusive end time since unix epoch [ns]
cut_basic	bool	Main S1 and main S2 (> s2_min_PE) present
cut_iso_s1	bool	No large alternative S1
cut_iso_s2	bool	No alternative S2 >= 5 % of the main S2
cut_s1_aft	bool	Main S1 AFT in the band of its drift time
cut_s2_width	bool	Main S2 width in the band of its drift time
cut_c	bool	Odd-event cut C (S2 AFT and width against the set's references)
cut_fiducial	bool	Main site inside the fiducial volume
cut_all	bool	All cuts
has_references	bool	Cut references available for the run

12. Loading

ev = st.get_df(run, ['events', 'event_cuts']) — one frame (same data kind). **m = amstrax.main_interaction(st, run)** — one flat row per event: the events columns, s1_area, s1_area_fraction_top, s1_range_50p_area, s1_n_channels, s1_tight_coincidence, drift_time, x, y, z, r, site_cs2, site_cs2_top, site_cs2_bottom, site_in_drift, s2_area, s2_area_fraction_top, s2_range_50p_area, s2_range_90p_area. **st.get_df(run, 'sites')**, **'event_peaks'**, **['peak_basics', 'peak_context']** — joined by key. Waveforms and per-channel areas: **st.get_array(run, 'peaks')[peak_index]** (full run). Worked examples (spectrum, multiple scatters, all peaks of an event, delayed signals, waveforms, porting old code), checked on run 007790: XAMS handbook, Data processing → Data format: use cases.

13. The old event format (amstrax ≤ 2.2, PROD0–PROD4)

Until 2026-11-06 the event level was one wide row per event: events (the windows) → event_basics (main and alternative S1 / S2 with their peak properties copied into the row, drift time) → event_positions (x, y, r, phi, z, u, v of the main S2) → corrected_areas (cs1, cs2 and cs2_top of the main S2, alt_cs1 / alt_cs2, energy) → event_info (stored merge of the three, 98 columns), plus event_area_per_channel, event_waveform and the NaI coincidences. Data processed then keep these types on disk; the current amstrax cannot load them. Read them with the frozen environment of tag prod4-final: **source /data/xenon/xams_v2/setup_prod4.sh**, then st.get_df(run, 'event_info') with corrections_version='PROD4' (reading only, never new processing). All old columns: XAMS handbook, Data format: old format.

event_info (amstrax ≤ 2.2)	amstrax 3	note
time, endtime, event_number, n_peaks	events (same names)	same windows for the same settings
cs1	events.cs1	same for one site; with more sites the S1 response is the cs2_top-weighted mean
cs2, cs2_top	main site: sites.cs2 / cs2_top of rank 1 (main_interaction: site_cs2, site_cs2_top)	events.cs2, cs2_top are **sums over all in-drift sites**
energy	events.energy	from the sums; the same for one site
alt_cs1 / alt_cs2	— / sites.cs2 of rank 2	alt S1: event_peaks row, uncorrected; alt S2 only if > 300 PE
drift_time, x, y, z, r (and s2_x, s2_y)	sites of rank 1 (main_interaction: same names)	every site has them
phi	arctan2(y, x)	not stored
u, v, s2_x_cgr, s2_y_cgr	peak_positions[peak_index of the main S2]	not in the event tables
s1_* / s2_* peak properties	event_peaks row of the main S1 / S2 (main_interaction: s1_area, s1_area_fraction_top, s1_range_50p_area, s1_n_channels, s1_tight_coincidence, s2_area, s2_area_fraction_top, s2_range_50p_area, s2_range_90p_area)	the rest from event_peaks, without prefix
alt_s1_*, alt_s2_*	event_peaks rows (rank, dt_s1)	second largest made explicit by rank
s1_index, s2_index	events.s1_peak_index, sites.peak_index	**changed meaning**: row in the run, not position in the event
s1_max_diff, s1_min_diff	peaks[peak_index].max_diff / min_diff	
area_before_main_s2, large_s2_before_main_s2	events (same names)	
event_area_per_channel, event_waveform	peaks[peak_index]: area_per_channel, data, data_top	removed types
peak_coincidences, event_coincidences	—	removed: match peak_basics_ext in time
events (windows); events_v2 (2.x optional)	event_windows; events	renamed in 3.0

Checked on run 007790 (173 651 events in both formats, the same event numbers): for single-site events drift_time, x, y of the main site equal those of event_info and z agrees within 0.1 %; cs1, cs2 and energy differ by 0.02–0.04 % (median), from the corrections (PROD4 → PROD5), not from the format.